

Multi Modal Road Assessment

Ashutosh Narayan Srivastava
*Electronics and Communication Engineering
Faculty of Technology, University of Delhi*

Abstract—Urban road networks face a severe condition-monitoring deficit due to the prohibitive cost of laser profilometry. While smartphone-based crowdsensing offers a scalable alternative, existing systems suffer from three fundamental limitations: speed-dependent signal distortion (the “staircase artifact”), static maps lacking temporal decay, and the absence of end-to-end multimodal routing integration. This paper presents a complete, deployed system resolving these limitations. To eliminate suspension-bias, we derive deterministic International Roughness Index (IRI) ground truth from the BeamNG.tech soft-body physics engine, extracting synchronised 100 Hz dual-LiDAR and 6-DoF IMU data from an 80 km/h Quarter-Car simulation. We structurally eliminate speed dependency via a dual-domain wavelet pipeline—applying a 1.11 Hz time-domain low-pass filter and a 5.4–25.0 m spatial-domain band-pass filter—to re-index signals onto a uniform 1 cm spatial grid. A lightweight, dual-input deep neural network (11,853 parameters) fuses a depthwise 1D CNN spatial extractor with a contextual MLP, trained using a Huber/Log-Cosh loss and inverse-class-frequency weighting to capture severe anomalies. Deployed via on-device TensorFlow Lite, the model achieves an MAE of 0.33 m/km. Our distributed Node.js backend aggregates observations with an exponential time decay ($\tau = 24$ hours) for self-healing map consistency, broadcasting sub-second updates via Geohash-precision-6 Web-Sockets. Finally, we introduce a multimodal routing service that scores candidate routes using a distance-weighted IRI composite, presenting users with a ranked multi-route comparison and a longitudinal personal Ride Comfort Score. To our knowledge, this is the first system unifying spatial-domain signal processing, simulation-derived ground truth, temporally adaptive crowdsourcing, and ride-comfort-aware navigation.

Index Terms—International Roughness Index, road surface monitoring, crowdsourcing, smartphone sensing, spatial domain processing, 1D convolutional neural network, real-time systems, infrastructure-aware routing, ride comfort score, temporal decay, Geohashing, BeamNG simulation.

I. INTRODUCTION

A. The Global Road Deterioration Problem

Road pavement is the most capital-intensive component of urban infrastructure. In India alone, the road network spans over 6.3 million kilometres [1], yet a large fraction of this network—particularly secondary and tertiary roads—receives structural condition assessment no more frequently than once every three to five years, if at all [1]. The World Bank estimates that the cumulative economic cost of poor road condition to vehicle owners exceeds **\$500 billion annually** worldwide, through accelerated tyre wear, fuel overconsumption, drivetrain damage, and road accident costs attributable to surface defects [2]. At the extremes of the spectrum, potholes—full-depth structural failures caused by the interaction of water infiltration, traffic loading, and thermal cycling—are respon-

sible for thousands of vehicle damage incidents and road accidents annually, and their formation can progress from surface cracking to a full pothole within a matter of weeks under heavy precipitation [].

The international standard for quantifying road roughness is the **International Roughness Index (IRI)**, defined by the World Bank Quarter-Car model and standardised in ISO 8608 and ASTM E1926 [3]. IRI represents the cumulative vertical displacement of a standardised quarter-car suspension traversing a road at 80 km/h, expressed in metres per kilometre (m/km). Professional measurement uses inertial laser profilometry vehicles whose procurement and operational costs are prohibitive for routine city-scale deployment. This creates a fundamental monitoring gap: the condition of the road network degrades continuously between surveys, and defects go unreported for months.

The challenge of closing this monitoring gap—with high spatial resolution, at low cost, and in near-real-time—motivates this work.

B. The Smartphone Sensing Opportunity

Every smartphone in use today is a capable mobile sensor platform: it contains a multi-axis MEMS inertial measurement unit (IMU), a GNSS receiver, and a persistent mobile data connection. With over 6.8 billion smartphones in global use [?], any vehicle in which a smartphone is mounted constitutes a potential road condition probe. If even a small fraction of the daily vehicle trips on a road network contribute passive IMU measurements, the aggregate creates a continuously updated, spatially dense picture of road surface condition at near-zero marginal cost—a paradigm known as **participatory or opportunistic sensing** [4].

This vision has motivated a growing body of research since Mohan et al.’s foundational Nericell system (2008) [5]. However, despite more than fifteen years of effort, no existing system achieves the full promise of this paradigm. Three fundamental limitations persistently appear across the literature and remain unresolved.

C. Three Unsolved Limitations of the State of the Art

Limitation 1 — Speed-Dependent Signal Distortion and the Staircase Artifact.

IMU sensors produce measurements as a function of time. Road roughness, however, is a geometric property of a spatial surface. The relationship between the two is mediated by vehicle speed v : a surface undulation of spatial wavelength λ (metres) produces a temporal frequency $f = v/\lambda$ in the

IMU signal. This means that the **same physical road feature appears at different temporal frequencies at different speeds**. At $v = 10$ km/h, a 5-metre undulation appears at ≈ 0.56 Hz; at $v = 50$ km/h, the same feature appears at ≈ 2.78 Hz—a five-fold shift. Any time-domain filter or time-domain machine learning model (including convolutional neural networks applied to raw IMU time series) is therefore inherently speed-dependent.

The practical consequence is the “staircase artifact”: at locations where vehicle speed changes abruptly—intersections, pedestrian crossings, traffic lights, roundabouts—time-domain processing pipelines produce discontinuous, step-change jumps in estimated road quality even on physically uniform surfaces. This is because the pipeline’s effective spatial frequency resolution changes with speed, and the transition appears as a sudden change in the measured roughness signal. Mukherjee et al. [6] acknowledge speed as a confounding factor and apply post-hoc normalisation; Sattar et al. [7] restrict experiments to highway driving at near-constant speed; the comprehensive review by the MDPI 2024 survey [8] identifies speed variation as one of the two primary open challenges in the field. No prior system has structurally eliminated this artifact through a full spatial-domain signal processing pipeline.

Limitation 2 — No Temporal Road State Evolution (The Stale Map Problem).

A road is not a static object. Its condition evolves continuously: a pothole that appears in February after winter frost-damage may be repaired by the municipal authority in March. A smooth surface may develop cracking by summer under heavy truck traffic. Any crowdsourced road condition map must model this temporal evolution—otherwise, the map becomes progressively stale and misleading.

Yet no existing smartphone-based road monitoring system implements a principled temporal decay model. Once a defect observation is stored, it persists indefinitely. Citizen reporting platforms such as SeeClickFix and FixMyStreet require explicit administrator action to close a report [9]. Academic systems including iDriveSense [10] and Roadroid [11] store observations without temporal weighting. Waze’s pothole reporting, evaluated by Goodall (2023) [9], clears reports after 30 minutes unless confirmed—a heuristic that bears no relationship to the actual timescale of road repair. The result is that in all prior systems, **a road repaired today will still show as damaged on the map for an indeterminate period**, severely degrading user trust and map utility.

Limitation 3 — Absence of End-to-End Multimodal Navigation Integration.

The ultimate utility of a road condition sensing system is not just awareness—it is the ability to act on that awareness. A driver who knows that Route A has a pothole cluster 2 km ahead should be able to choose Route B, with full knowledge of the trade-off between the extra distance and the improved ride quality. This is the multi-criteria route planning problem: finding a route that optimises a composite objective of travel distance, travel time, and infrastructure quality.

Despite this obvious utility, no existing peer-reviewed system delivers a complete, live, navigation-integrated deployment in which road quality derived from crowdsourced smartphone sensing is used as a first-class routing criterion. iDriveSense [10] proposes a fuzzy-logic route scoring mechanism but relies on manually annotated road quality maps rather than a continuously updated crowdsourced layer, and is not deployed as a working mobile application. The system of Dewangan et al. [12] and the pothole avoidance framework of Balakuntala et al. [13] are conceptual frameworks without real-time backend integration. Commercial navigation systems (Google Maps, HERE, Waze) do not incorporate independently computed IRI-based quality scores. This gap—between a working crowdsourced sensing backend and a real, multimodal navigation experience for the end user—represents the most practically impactful limitation in the field.

D. Additional Gaps in Prior Work

Beyond the three primary limitations, a systematic review of the literature reveals several further recurring weaknesses:

Vehicle-Suspension Coupling and the Cross-Vehicle Generalisation Problem. Virtually all empirical training datasets are collected from a single vehicle or a small number of homogeneous vehicles. The suspension system of a rigid heavy vehicle filters road surface inputs very differently from a soft-sprung passenger car: the same road profile produces different IMU signal amplitudes, damping profiles, and resonant frequencies across vehicle classes. A model trained on data from one vehicle type and deployed on another exhibits systematic, uncorrected bias [6], [7]. Only a handful of works attempt cross-vehicle normalisation, and none solve the problem from first principles.

Absence of Physically Calibrated Ground Truth. The majority of published systems label their training data with subjective human ratings (good/fair/poor), manual event annotations, or binary thresholds on the z-axis accelerometer [14], [11]. These labels are not reproducible, not physically calibrated, and not inter-comparable across studies. The IRI is the internationally standardised metric of road roughness [3], but obtaining true IRI ground truth in the field requires co-registered laser profilometry—a logistical challenge that most papers do not address. This absence of calibrated ground truth makes rigorous comparison between published systems impossible.

Computational Deployability. Systems that achieve high detection accuracy often do so at the cost of models too large or computationally expensive for continuous background inference on a smartphone [8]. Large LSTM networks, multi-scale ResNets, and attention-based transformers may achieve strong performance on server-side batch evaluation but are impractical as always-on on-device sensors due to battery and thermal constraints.

No Personal Ride History or Longitudinal Comfort Analytics. No existing system maintains a per-user ride history that allows individuals to understand their cumulative road experience over time—which routes they regularly travel, how

the quality of those routes has evolved, and what their average Ride Comfort Score is. This longitudinal dimension is entirely absent from the academic literature, despite its obvious value for urban mobility analytics and municipal reporting.

E. Contributions of This Paper

This paper presents a complete, deployed, end-to-end system for real-time crowdsourced road surface evaluation and multimodal navigation. Our specific contributions, each addressing one of the gaps identified above, are as follows:

- 1) **Speed-invariant spatial-domain signal processing.** We introduce a dual-domain wavelet pipeline that rigorously transforms all IMU and LiDAR signals from the irregular time domain onto a uniform 1 cm spatial grid, applying a 1.11 Hz time-domain low-pass filter for chassis resonance suppression and a 5.4–25.0 m spatial-domain band-pass filter for macro-slope removal. This structural reformulation eliminates speed dependency and the staircase artifact.
- 2) **Physics-simulation-derived, vehicle-agnostic ground truth via BeamNG.tech.** We exploit the BeamNG.tech soft-body physics simulator—the same platform used for ADAS research and autonomous driving validation [15]—to generate synchronised (IMU, LiDAR profile) pairs across multiple vehicle models at the ISO-standard 80 km/h, providing deterministic, noise-free IRI labels that cannot be practically obtained from field surveys.
- 3) **Dual-input deep neural network with physically motivated architecture.** We propose an 11,853-parameter model combining depthwise-separable 1D convolutions (for per-channel spatial feature extraction) with a contextual MLP (for speed- and vehicle-type-conditioned regression), trained under a composite Huber/Log-Cosh loss function with inverse-class-frequency pothole weighting and a false-alarm suppression mechanism. The model is deployed on Android via TensorFlow Lite post-training quantisation.
- 4) **Temporally adaptive crowdsourced map with self-healing consistency.** We implement an exponential time-decay aggregation model ($w \propto e^{-t/\tau}$) in which every observation’s contribution to a road segment’s quality score diminishes with age. This enables repaired road defects to be automatically and continuously down-weighted without any manual intervention—a property we term **self-healing map consistency**, not previously described in the literature.
- 5) **Sub-second real-time broadcast via Geohash Web-Socket namespacing.** The distributed backend routes road quality events exclusively to clients whose geographic viewport overlaps the affected area, using precision-6 Geohash strings as Socket.IO room identifiers. This achieves $O(R)$ broadcast complexity (R = clients in a $\approx 1.2 \text{ km} \times 0.61 \text{ km}$ cell) without global broadcast overhead.
- 6) **Multimodal ride-comfort-aware routing.** A routing service retrieves multiple candidate routes from the

Mapbox Directions API, scores each by a distance-weighted composite of IRI-based road quality and route length, and presents the user with a ranked multi-route comparison. Users can select their preferred balance between ride comfort and distance—directly addressing the multi-criteria route planning gap identified above.

- 7) **Personal Ride Comfort Score and trip history.** Each completed journey is stored as a user-associated record containing the route geometry, mean IRI, and an overall Ride Comfort Score derived from the spatial quality distribution of the traversed road. This enables longitudinal personal mobility analytics and provides municipalities with user-linked pavement condition reports.

F. Paper Organisation

Section II surveys related work across road surface monitoring, crowdsourcing architectures, and infrastructure-aware routing. Section III details the complete methodology and system architecture. Section IV presents experimental results. Section V discusses current limitations and future work—including the Pothole Classification Model under active development. Section VI concludes.

II. RELATED WORK

A. Traditional Road Condition Assessment

The established method for road condition assessment is the **inertial laser profilometer**: a vehicle-mounted system that uses laser displacement sensors and a high-precision IMU to measure the road surface profile at driving speeds of 60–100 km/h, computing IRI in accordance with ASTM E1926 [?]. These systems achieve high accuracy (IRI precision $\pm 0.1 \text{ m/km}$) but cost upward of \$200,000 USD per vehicle, require skilled operators, and are logistically suited only to periodic surveys—not continuous monitoring. Some agencies use Response Type Road Roughness Measurement Systems (RTRRMS), such as bump integrators mounted in standardised vehicles, which are cheaper but provide lower accuracy and require correlation calibration [?].

Ground-penetrating radar, 3D LiDAR scanning, and photogrammetric inspection from unmanned aerial vehicles (UAVs) have been proposed for crack detection and structural assessment [?]; these methods provide rich surface geometry but at high per-km cost and low temporal frequency, making city-scale continuous monitoring impractical.

III. METHODOLOGY AND SYSTEM ARCHITECTURE

A. Synthetic Data Acquisition and Ground Truth Generation

1) *The Fundamental Problem with Empirical Datasets:* A recurring limitation in prior smartphone-based road profiling literature is the reliance on empirical, field-collected datasets for model training and ground truth establishment [?]. While seemingly natural, this approach introduces a profound and often underappreciated confound: **the physical coupling between the road surface and the IMU sensor is not fixed.** It is mediated by the vehicle’s suspension system—a highly

nonlinear, vehicle-specific, and load-dependent mechanical transfer function.

Formally, if we denote the true road profile as $h(x)$ and the sensor-observed vertical acceleration as $a_z(t)$, the empirical relationship is:

$$a_z(t) = \mathcal{T}_{\text{veh}}(h(x(t)), \dot{x}(t), m_{\text{load}}, T_{\text{ambient}}) + \epsilon(t) \quad (1)$$

where $\mathcal{T}_{\text{veh}}$ is an unknown, nonlinear operator characterizing the vehicle’s full suspension dynamics, $\dot{x}(t)$ is the instantaneous speed, m_{load} is the payload mass, T_{ambient} is the ambient temperature (which affects tire pressure and damper viscosity), and $\epsilon(t)$ is sensor noise. This operator is not only **unknown** for any given vehicle but is **different across vehicles**. A model trained on a dataset collected in a mid-size sedan will exhibit systematic bias when deployed on a utility vehicle with stiffer suspension—even on identical roads—because $\mathcal{T}_{\text{veh}}$ differs fundamentally between the two platforms.

Furthermore, empirical field collection makes it practically impossible to obtain a true, decoupled ground truth for $h(x)$. Laser profilometers or rod-and-level instruments can provide such ground truth, but co-registering their spatial measurements with the corresponding IMU time series across diverse road conditions, vehicle speeds, and vehicle types is logistically intractable at the scale required to train a robust deep learning model.

This paper directly resolves both problems through the use of a physics simulation environment.

2) *The BeamNG.tech Soft-Body Physics Engine as a Deterministic Laboratory*: We employ **BeamNG.tech** (v0.38.3.0), a professional-grade vehicular dynamics simulator based on a soft-body physics model, as our synthetic data generation environment. This is a fundamental departure from prior work. The simulation’s physical fidelity is sufficient for automotive engineering research precisely because it models vehicles not as rigid bodies with simplified spring-damper suspension proxies, but as fully deformable finite-element meshes where every node participates in the physical simulation. The tire-road interaction, damper hysteresis, and chassis flex are all modeled from first principles.

The critical epistemological advantage of this approach is **Oracle Access to Ground Truth**. Within the simulation, we have simultaneous, synchronized, zero-noise access to:

- 1) **The true road surface geometry** $h(x)$, obtained directly via dual downward-facing LiDAR sensors mounted at the left and right wheel tracks.
- 2) **The vehicle-filtered IMU response** $(a_x, a_y, a_z, \omega_x, \omega_y, \omega_z)$, obtained from a virtual AdvancedIMU sensor positioned at the vehicle dashboard—precisely replicating a smartphone in a dashboard mount.
- 3) **The vehicle’s true kinematic state**, including instantaneous 3D velocity (v_x, v_y, v_z) and world-space position (p_x, p_y, p_z) .

The code reflects this three-thread architecture explicitly. A `bng_lock` serializes all communication with the physics

engine, while a `data_lock` protects the `shared_data` dictionary that is written by sensor worker threads and consumed by the main logging loop. Three daemon threads run in parallel:

- **ImuStateWorker**: Polls the virtual AdvancedIMU sensor and vehicle state at $f_{\text{IMU}} = 100$ Hz, extracting and re-mapping acceleration and angular velocity axes to match real-world smartphone conventions (BeamNG’s native coordinate system differs from the standard right-hand IMU frame).
- **LidarWorker**: Polls two narrow-beam downward-facing LiDAR sensors (mounted at ± 0.85 m lateral offset, approximating wheel-track positions) at $f_{\text{IRI}} = 100$ Hz, extracting the mean Z-elevation of the point cloud to yield instantaneous road surface height $h_L(t)$ and $h_R(t)$.
- **CameraWorker**: Captures a forward-facing dashcam view at $f_{\text{CAM}} = 20$ Hz for qualitative inspection.

The critical coordinate re-mapping performed in `ImuStateWorker` deserves explicit documentation, as it is an often-silently-introduced source of error in simulation-to-real transfer:

TABLE I
COORDINATE RE-MAPPING FROM BEAMNG TO PHYSICAL IMU FRAME

BeamNG Raw Channel	Physical Axis	Mapped to <code>shared_data</code>
<code>acc[2]</code> (Lateral, Y-world)	Left-Right	<code>ax</code>
<code>acc[0]</code> (Longitudinal, X-world)	Front-Back	<code>ay</code>
<code>acc[1]</code> (Vertical, Z-world)	Up-Down	<code>az</code>
<code>gyro[2]</code> (Pitch rate)	Around X-lateral	<code>wx</code>
<code>gyro[0]</code> (Roll rate)	Around Y-longitudinal	<code>wy</code>
<code>gyro[1]</code> (Yaw rate)	Around Z-vertical	<code>wz</code>

The main logging loop runs as a **strict metronome** using `time.perf_counter()`, enforcing a precisely $\Delta t = 10$ ms cadence between samples, independent of any individual sensor’s polling latency. This ensures the CSV log has uniform temporal spacing—a prerequisite for the spatial transformation pipeline described in Section III-B.

The simulation runs with a simulated vehicle speed of $v_{\text{sim}} = 80$ km/h, corresponding to the **ISO 8608 and World Bank IRC standard speed** for IRI computation. This is not an arbitrary choice; the IRI standard is defined with respect to this speed precisely because the quarter-car system is calibrated to resonate at frequencies characteristic of surface irregularities encountered at 80 km/h, yielding a dimensionally meaningful roughness measure.

We collect data across three distinct virtual maps—`east_coast_usa` (hilly terrain with unpaved sections), `automation_test_track` (controlled circuit with varied surface types), and `west_coast_usa` (highway and urban arterials)—and across three vehicle models (`hopper`, `sunburst2`, `vivace`) representing a range of suspension stiffnesses. This multi-vehicle, multi-environment corpus ensures that the learned model generalizes across the suspension-coupling variability that plagues empirical approaches.

B. Signal Processing and Spatial Domain Translation

1) The Fundamental Flaw of Time-Domain Sensor Data:

The single most consequential—and most commonly ignored—error in smartphone-based road roughness estimation is **the speed-distortion of the time domain**. Every sensor onboard a smartphone produces data as a function of time t . However, road roughness is a property of the road surface—it is a function of *position along the road*, not of time. This distinction is not merely semantic; it has direct and destructive consequences for signal processing and machine learning.

Consider a road with a periodic surface undulation of spatial period $\lambda = 5$ m. When traversed at $v_1 = 10$ km/h, this undulation appears in the IMU signal at temporal frequency:

$$f_1 = \frac{v_1}{\lambda} = \frac{10/3.6}{5} \approx 0.56 \text{ Hz} \quad (2)$$

When the same stretch of road is traversed at $v_2 = 50$ km/h, the same undulation appears at:

$$f_2 = \frac{v_2}{\lambda} = \frac{50/3.6}{5} \approx 2.78 \text{ Hz} \quad (3)$$

A time-domain filter designed to isolate road roughness at one speed will either pass or attenuate the same physical feature at a different speed. This is the root cause of the "speed dependency" problem reported across the literature [?]. Time-domain convolutional networks suffer from the same pathology: a CNN trained on data at predominantly high speeds will learn temporal filter kernels tuned to high temporal frequencies, and will fail on low-speed data where the same physical feature manifests at a lower frequency. The converse is equally true.

The solution, which we implement rigorously, is to **abandon the time domain entirely** and operate exclusively in the spatial domain.

2) *The Spatial Domain Transformation:* Let the vehicle's instantaneous speed at sample k be v_k (in m/s), and the uniform inter-sample time be $\Delta t = 0.01$ s ($f = 100$ Hz). The infinitesimal displacement in the sensor frame at each timestep is:

$$dx_k = v_k \cdot \Delta t \quad (4)$$

The cumulative distance travelled after N samples is then:

$$d_N = \sum_{k=0}^{N-1} v_k \cdot \Delta t \quad (5)$$

In our code, the speed v_k is derived from the 3D velocity vector:

$$v_k = \sqrt{v_{x,k}^2 + v_{y,k}^2 + v_{z,k}^2} \quad (6)$$

In the processing pipeline (`iri_simple.py`), we compute the cumulative distance more precisely using the actual Euclidean distance between successive 3D world-space positions:

$$d_N = \sum_{k=1}^N \sqrt{(\Delta p_{x,k})^2 + (\Delta p_{y,k})^2 + (\Delta p_{z,k})^2} \quad (7)$$

This accounts for genuine 3D motion (e.g., a vehicle climbing a hill) rather than projecting purely onto a 2D map plane, which would introduce systematic underestimation of path length on steep grades.

Once the cumulative distance array $\{d_k\}$ is established, we re-index all sensor signals from the irregular, speed-dependent time basis onto a **uniform spatial grid** with spacing $\Delta x = 1$ cm:

$$\mathcal{G} = \{0, \Delta x, 2\Delta x, \dots, \lfloor d_{\max}/\Delta x \rfloor \cdot \Delta x\} \quad (8)$$

This is accomplished via cubic spline interpolation: for each signal channel s with time-domain values $\{s_k\}$ at cumulative distances $\{d_k\}$, we construct the interpolant $\hat{s}(d)$ and evaluate it on $\mathcal{G}$. After this transform, all signals are functions of distance—a physically invariant coordinate—not of time. A 5-metre road undulation now occupies exactly 500 spatial samples regardless of the speed at which it was traversed.

This transformation is the critical enabler for the speed-invariant signal processing and deep learning pipeline that follows.

3) *Dual-Domain Wavelet Processing:* The elevation signals $h_L(d)$ and $h_R(d)$, now in the spatial domain, contain three classes of contamination that must be removed before IRI computation:

- 1) **Chassis Resonance and Suspension Oscillation (High-frequency, Time-Domain):** The vehicle's suspension system does not transmit the road surface faithfully. Low-frequency chassis body bounce (~ 1 – 2 Hz) and high-frequency suspension resonances pollute the LiDAR-measured elevation with spurious oscillations that are artifacts of the mechanical system, not the road surface. These artifacts are correlated with speed.
- 2) **Macro-scale Grade and Hilliness (Low-frequency, Spatial-Domain):** A road may traverse hills, overpasses, or gradients. These "DC" and very-low-frequency spatial components represent the macro-profile of the road—its shape at scales of tens of meters—not the micro-profile (roughness) we seek to measure.
- 3) **Sub-IRI Texture (Very high-frequency, Spatial-Domain):** Surface texture at spatial frequencies shorter than ~ 0.25 m (pebbles, aggregate, etc.) is not sensed by a vehicle's suspension and thus does not contribute to the IRI metric. Its inclusion would constitute noise.

We address these three contaminations via a two-stage, dual-domain wavelet filtering strategy, using the **Daubechies 4 (db4) wavelet** throughout, chosen for its compact support, orthogonality, and high approximation order, which preserves signal energy at the scales of interest.

Stage 1: Time-Domain Wavelet Low-Pass Filter (Chassis Resonance Mitigation)

Prior to spatial transformation, the raw LiDAR elevation time series $h_L(t)$ undergoes a wavelet-domain low-pass filter with cutoff $f_c = 1.11$ Hz, which corresponds to the approximate lower bound of chassis resonance frequency.

The decomposition level L is chosen adaptively from the sampling frequency f_s :

$$L = \left\lfloor \log_2 \left(\frac{f_s}{f_c} \right) \right\rfloor - 1 \quad (9)$$

At $f_s = 100$ Hz and $f_c = 1.11$ Hz, this yields $L \approx 5$. The wavelet DWT is computed:

$$\{cA_L, cD_L, cD_{L-1}, \dots, cD_1\} = \text{DWT}^L(h(t)) \quad (10)$$

The low-pass filter retains only the approximation coefficients cA_L (representing components below f_c) and zeros all detail coefficients:

$$\{cA_L, \mathbf{0}, \mathbf{0}, \dots, \mathbf{0}\} \xrightarrow{\text{IDWT}} \hat{h}_{\text{LPF}}(t) \quad (11)$$

The resulting signal $\hat{h}_{\text{LPF}}(t)$ retains the macro-shape of the road surface as seen by the LiDAR, while suppressing the rapid oscillations induced by chassis bounce and sensor vibration.

Stage 2: Spatial-Domain Wavelet Band-Pass Filter (Macro-Hill Slope Removal)

After spatial transformation, the elevation profile $\hat{h}_{\text{LPF}}(d)$ undergoes a spatial band-pass filter. We wish to retain only spatial wavelengths λ in the range:

$$\lambda \in [5.4 \text{ m}, 25.0 \text{ m}] \quad (12)$$

This range is chosen to align with the transfer function sensitivity of the Quarter-Car golden car model (see Section III-B4): the golden car system predominantly integrates surface irregularities in this wavelength band at $v = 80$ km/h. The lower bound suppresses micro-texture; the upper bound suppresses hill slopes and bridge approaches.

The spatial band-pass is implemented as a wavelet decomposition of depth:

$$L_{\text{decomp}} = \max \left(\left\lceil \log_2 \left(\frac{\lambda_{\text{max}}}{2\Delta x} \right) \right\rceil + 1, 12 \right) \quad (13)$$

At $\Delta x = 0.01$ m, the target wavelet decomposition levels corresponding to the band boundaries are:

$$L_{\text{min}} = \left\lceil \log_2 \left(\frac{\lambda_{\text{min}}}{2\Delta x} \right) \right\rceil = \left\lceil \log_2 \left(\frac{5.4}{0.02} \right) \right\rceil \approx 8 \quad (14)$$

$$L_{\text{max}} = \left\lceil \log_2 \left(\frac{\lambda_{\text{max}}}{2\Delta x} \right) \right\rceil = \left\lceil \log_2 \left(\frac{25}{0.02} \right) \right\rceil \approx 10 \quad (15)$$

The reconstruction retains only detail coefficients at levels $L \in [L_{\text{min}}, L_{\text{max}}]$:

$$z_{\text{BPF}}(d) = \text{IDWT}(\mathbf{0}, \{cD_L \cdot \mathbf{1}[L_{\text{min}} \leq L \leq L_{\text{max}}]\}) \quad (16)$$

Prior to this band-pass, the spatial profile is linearly detrended via `scipy.signal.detrend`, which removes any remaining linear grade component that the band-pass might not fully eliminate. The combination of linear detrending and wavelet band-pass together constitutes a robust high-order spatial filter that is immune to both macro-hilliness (the "staircase artifact" caused by naive fixed-threshold filtering in the time domain) and sub-texture noise.

Eliminating the Staircase Artifact

The "staircase artifact" is a well-documented failure mode in road profiling literature [?]. It arises when a time-domain filter with a fixed temporal cutoff frequency is applied to data collected at variable speeds: as speed changes abruptly (e.g., at intersections), the filter's effective spatial cutoff shifts, producing discontinuous step-changes in the estimated road quality score even on homogeneous surfaces. Our spatial domain reformulation **structurally eliminates this artifact**: because all filtering is performed on the spatially-re-indexed signal, a fixed spatial cutoff corresponds to the same physical scale regardless of the speed at which any particular road segment was traversed.

4) *The International Roughness Index via the Quarter-Car Model*: The IRI is computed from the processed spatial profile using the **ISO 8608 / World Bank Quarter-Car (Golden Car)** linear dynamical model. This model is a 2-degree-of-freedom representation of a standardized quarter-car suspension system, parameterized with values established by the World Bank to maximize correlation with human ride perception across diverse road types. The canonical parameters are:

TABLE II
CANONICAL QUARTER-CAR (GOLDEN CAR) PARAMETERS

Parameter	Symbol	Value	Physical Meaning
Damping coefficient	C	6.0	Suspension damping
Unsprung-sprung stiffness	K_1	63.3	Suspension spring
Tire stiffness	K_2	653.0	Tire spring
Unsprung-to-sprung mass ratio	μ	0.15	Mass ratio

The state vector $\mathbf{x} = [z_s, \dot{z}_s, z_u, \dot{z}_u]^T$ collects the sprung-mass displacement, sprung-mass velocity, unsprung-mass displacement, and unsprung-mass velocity. The system matrices are:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -K_1 & -C & K_1 & C \\ 0 & 0 & 0 & 1 \\ K_1/\mu & C/\mu & -(K_1 + K_2)/\mu & -C/\mu \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ K_2/\mu \end{bmatrix} \quad (17)$$

$$\mathbf{C}_{\text{out}} = [0 \quad 1 \quad 0 \quad -1], \quad \mathbf{D} = \mathbf{0} \quad (18)$$

The output $y(t) = \mathbf{C}_{\text{out}}\mathbf{x}(t)$ is the *relative velocity* between the sprung and unsprung masses—the suspension stroke rate, which is the quantity whose integral defines the IRI.

To numerically integrate this system over a 1-meter road segment patch $z(d)$, we convert the spatial input to a time-domain input by the substitution $t = d/v_{sim}$, yielding sample interval $dt = \Delta x/v_{sim} = 0.01/22.22 \approx 4.5 \times 10^{-4}$ s, and invoke `scipy.signal.lsim` on the `StateSpace` system object.

The IRI for a segment of length L_{seg} (in km) is then:

$$IRI = \frac{1}{L_{seg}} \int_0^{L_{seg}/v_{sim}} |y(t)| dt \approx \frac{dt}{L_{seg}} \sum_k |y_k| \quad (19)$$

This quantity, expressed in m/km, is the universal, standardized measure of road roughness adopted by the World Bank, ASTM, and ISO. The averaged left and right IRI:

$$IRI_{avg} = \frac{IRI_L + IRI_R}{2} \quad (20)$$

constitutes the target label for each 1-metre patch in our training dataset. The `readings.csv` output file maps each IMU sample back to its corresponding 1-metre patch IRI via spatial binning—establishing the one-to-one correspondence between raw sensor signal windows and ground truth labels that supervises the deep learning pipeline.

C. Dual-Input Deep Learning Architecture

1) Architectural Motivation and Input Representation:

The trained model must solve a regression problem: given a window of sensor data covering a fixed spatial extent of road, predict the IRI of that road segment. Two classes of features are relevant:

- 1) **Micro-scale kinematic geometry:** The shape of the acceleration and angular velocity signals across the spatial window encodes the road’s roughness profile as filtered through the vehicle’s suspension. These features are intrinsically *sequential*—their spatial ordering carries information (a specific oscillation pattern, a sharp impulse followed by damping).
- 2) **Macro-scale contextual statistics:** The mean vehicle speed, the mean and variance of vertical acceleration, the zero-crossing rate of the vertical signal, and a one-hot encoding of the vehicle type modulate how the micro-scale signals should be interpreted. A given amplitude of vertical acceleration means different things at 10 km/h versus 60 km/h, or in a rigid truck versus a soft-sprung sedan.

These two classes of features have fundamentally different mathematical structures: one is a multivariate time series (amenable to convolutional processing); the other is a fixed-length vector of global statistics (amenable to dense/MLP processing). This motivates a **dual-head, fusion architecture**.

2) *Training Data Preparation: Spatial Windowing and Speed Augmentation:* Before model training, each trip’s `readings.csv` is processed by a parallel pipeline that converts point-sample readings into **spatially-fixed windows**. For each window:

- **Window extent:** 5.0 m in the spatial domain, with a 2.0 m stride (producing overlapping windows for data augmentation).
- **Spatial grid:** Each window is sub-sampled or interpolated to exactly $N = 100$ uniformly-spaced spatial positions via linear interpolation, regardless of how many raw IMU samples fell within that window.
- **Target label:** The mean IRI across all 1-metre patches within the window’s spatial extent.

A key innovation in the data pipeline is **speed augmentation**. The raw BeamNG simulation produces data at a fixed simulated speed, but the deployed model must be robust to the real-world range of vehicle speeds (5–30 km/h). We simulate this by sub-sampling the IMU data within each window at six different effective polling rates: $\{5, 10, 15, 20, 25, 30\}$ Hz—representing the effective IMU capture rates at different driving speeds. For each polling rate, we:

- 1) Draw pseudo-random sample times with small jitter (± 20 ms) to simulate clock imprecision.
- 2) Interpolate feature values onto these irregular sample times using `scipy.interpolate.interp1d` with `kind='previous'` (zero-order hold, matching real embedded sensor behavior).
- 3) Re-interpolate the resulting irregularly-sampled spatial sequence back onto a uniform $N = 100$ -point grid.

The result is that each 5-metre window of road generates 6 training examples—each representing the same road at a different effective sensor rate/speed combination. This augmentation is the mechanism by which the model learns speed-invariant features.

The seven input channels per window are $[\hat{a}_x, \hat{a}_y, \hat{a}_z, \hat{\omega}_x, \hat{\omega}_y, \hat{\omega}_z, v]$, where v is the instantaneous speed (included to provide the model with the physics context it needs to correctly scale the kinematic features).

3) *The Spatial Feature Extractor (1D CNN Branch):* The first input head receives the 100×7 spatial-kinematic tensor. We denote this input as $\mathbf{X}_{raw} \in \mathbb{R}^{100 \times 7}$.

Layer 1: Depthwise Conv1D (Channel Isolation)

The first convolutional layer is architecturally critical and departs from prior work. A standard `Conv1D` with 64 filters would, in its first layer, compute arbitrary linear combinations across all 7 input channels—immediately mixing the lateral (a_x), longitudinal (a_y), vertical (a_z), roll (ω_x), pitch (ω_y), yaw (ω_z), and speed (v) channels. This is physically wrong: the channels have fundamentally different units and different physical meanings. Mixing them in the first layer forces the network to simultaneously learn channel weights and temporal/spatial patterns—a harder optimization problem.

Instead, we use `DepthwiseConv1D` with `kernel_size=5` and `depth_multiplier=2`. A depthwise convolution applies a **separate** set of filters to each input channel independently:

$$y_c[n] = \sum_{k=0}^{K-1} W_c[k] \cdot X_{raw}[n+k, c], \quad c = 1, \dots, 7 \quad (21)$$

The output is $\mathbb{R}^{100 \times 14}$ (7 channels $\times$ 2 depth multiplier). This first layer learns *within-channel* spatial features—how each axis accelerates and decelerates across the window—without any cross-channel mixing. The number of parameters is minimal: $7 \times 5 \times 2 = 70$ weights, plus 14 biases (84 total), making this the lightest possible “feature extraction” first layer.

Layers 2–4: Standard Conv1D + Pooling

After batch normalization and max-pooling to $\mathbb{R}^{50 \times 14}$, a standard Conv1D(filters=64, kernel_size=3) is applied. Now that the channels have been individually characterized by the depthwise layer, the standard convolution can legitimately perform cross-channel mixing—learning correlations between, for instance, vertical acceleration and pitch rate that are diagnostic of a pothole traversal versus a speed-bump traversal.

After a second batch normalization and max-pooling to $\mathbb{R}^{25 \times 64}$, a GlobalMaxPooling1D layer extracts the most strongly activated spatial feature across the entire window, yielding a 64-dimensional feature vector $\mathbf{f}_{CNN} \in \mathbb{R}^{64}$.

The use of GlobalMaxPooling1D rather than GlobalAveragePooling1D is deliberate: maximum pooling captures the *peak* response to a perturbation event (e.g., a single pothole), which is a more stable and discriminative feature for roughness estimation than the average, which can be diluted by smooth sections within the window.

4) *The Contextual Network (MLP Branch)*: The second input head receives the 7-dimensional context vector:

$$\mathbf{X}_{ctx} = [\bar{v}, \bar{\hat{a}}_z, \bar{\hat{a}}_y, \text{MCR}_{a_z}, \mathbf{1}_{viv}, \mathbf{1}_{sun}, \mathbf{1}_{hop}] \quad (22)$$

where $\bar{v}$ is the mean speed over the window (m/s), $\bar{\hat{a}}_z$ is the mean filtered vertical acceleration, $\bar{\hat{a}}_y$ is the mean filtered longitudinal acceleration, MCR_{a_z} is the **mean zero-crossing rate of vertical acceleration** (a proxy for roughness frequency), and $[\mathbf{1}_{viv}, \mathbf{1}_{sun}, \mathbf{1}_{hop}]$ is a one-hot vehicle type indicator.

The mean speed $\bar{v}$ is particularly important: it provides the scale factor needed to interpret the kinematic signal amplitudes. At low speed, a given IRI produces smaller accelerations than at high speed. Without this information, the CNN branch would be forced to learn a family of responses parameterized by speed—a far harder problem than learning a single physics-agnostic roughness signature given the speed context.

A two-layer MLP processes this context:

$$\mathbf{f}_{ctx} = \text{Dropout}(\text{BN}(\text{ReLU}(\mathbf{W}_1 \mathbf{X}_{ctx} + \mathbf{b}_1))) \in \mathbb{R}^{32} \quad (23)$$

5) *Feature Fusion and Regression Head*: The CNN features $\mathbf{f}_{CNN} \in \mathbb{R}^{64}$ and context features $\mathbf{f}_{ctx} \in \mathbb{R}^{32}$ are concatenated into a 96-dimensional fusion vector:

$$\mathbf{f}_{fused} = [\mathbf{f}_{CNN}; \mathbf{f}_{ctx}] \in \mathbb{R}^{96} \quad (24)$$

Two further dense layers with ReLU activation, batch normalization, and dropout reduce this to a scalar IRI prediction:

$$\hat{y} = \text{ReLU}(\mathbf{w}_{out}^T \mathbf{z} + b_{out}) \quad (25)$$

The ReLU activation on the final output enforces non-negativity of the IRI prediction, which is physically constrained ($\text{IRI} \geq 0$).

The total parameter count of the model is **11,853** (11,633 trainable, 220 frozen batch normalization). This extremely compact footprint is essential for on-device deployment as a TensorFlow Lite model: the model is quantized via post-training default quantization (tf.lite.Optimize.DEFAULT), converting float32 weights to mixed integer/float16 representations, enabling real-time inference on mid-range Android processors with negligible accuracy degradation.

6) *The Custom Composite Loss Function: Huber + Log-Cosh with Adaptive Penalties*: Standard Mean Squared Error (MSE) loss, commonly used in regression, is theoretically optimal under the assumption that residuals $\epsilon = y - \hat{y}$ are i.i.d. Gaussian. For IRI prediction, this assumption is severely violated. IRI distributions over real road networks are **heavy-tailed and highly skewed**: the vast majority of road segments have IRI in the range 1–5 m/km (good to fair), while a small but critically important fraction exhibits IRI ≥ 8 m/km (severely damaged, pothole-ridden). Under MSE, these rare high-IRI examples contribute $(y - \hat{y})^2$ to the loss—a term that grows quadratically and can dominate the gradient signal, destabilizing training. Conversely, for moderately good roads, MSE provides gradients that are linearly proportional to error, which is appropriate.

We address these competing concerns with a **composite loss function**:

$$\mathcal{L}_{base}(\epsilon) = \mathcal{L}_{Huber}(\epsilon) + 0.2 \cdot \mathcal{L}_{LogCosh}(\epsilon) \quad (26)$$

where the **Huber loss** is:

$$\mathcal{L}_{Huber}(\epsilon) = \begin{cases} \frac{1}{2}\epsilon^2 & \text{if } |\epsilon| \leq \delta \\ \delta \left(|\epsilon| - \frac{\delta}{2} \right) & \text{if } |\epsilon| > \delta \end{cases} \quad (27)$$

with $\delta = 1.0$ m/km. This function is quadratic (MSE-like, optimal for Gaussian residuals) for errors within 1 m/km, and transitions to linear growth for larger errors—clipping the influence of anomalous outliers and providing gradient stability during early training on the skewed IRI distribution.

The **Log-Cosh loss** provides complementary behavior:

$$\mathcal{L}_{LogCosh}(\epsilon) = \log \cosh(\epsilon) \approx \begin{cases} \frac{1}{2}\epsilon^2 & \text{for small } |\epsilon| \\ |\epsilon| - \log 2 & \text{for large } |\epsilon| \end{cases} \quad (28)$$

The additive combination achieves twice-differentiability everywhere (important for second-order optimizer compatibility), with the logarithmic tail of Log-Cosh providing even heavier regularization of outliers than Huber alone.

The Pothole Penalty (Inverse Class Frequency Strategy)

To address the class imbalance—where high-IRI examples are rare but critical—we apply a multiplicative pothole penalty to the base loss:

$$w_{\text{pothole}}(y) = \begin{cases} 3.0 & \text{if } y > 4.0 \text{ m/km} \\ 1.0 & \text{otherwise} \end{cases} \quad (29)$$

This is a discrete implementation of **inverse class frequency weighting**: the effective gradient for a severely damaged road segment is tripled relative to a smooth road segment, compensating for the $\sim 3:1$ imbalance between well-represented smooth roads and rare severely damaged roads in the training distribution.

The False-Alarm Suppression Hunter

A separate failure mode is the model’s tendency to predict high IRI during vehicle acceleration/deceleration events or at very low speeds, even on smooth roads. These produce large longitudinal and vertical acceleration spikes that can confuse a naive model into predicting road damage when none exists. We suppress this via an explicit **false-alarm penalty**:

$$\text{FalseAlarm} = \mathbf{1}[y < 2.0] \wedge (\mathbf{1}[v < 5.56 \text{ m/s}] \vee \mathbf{1}[|\bar{a}_y| > 2.5 \text{ m/s}^2]) \wedge \mathbf{1}[\hat{y} > 2.0] \quad (30)$$

$$w_{\text{hunter}} = 1.0 + 5.0 \cdot \text{FalseAlarm} \quad (31)$$

When the road is objectively smooth (true $\text{IRI} < 2 \text{ m/km}$), the vehicle is either moving slowly or decelerating hard, and the model predicts high roughness, the gradient signal is quintupled—delivering a sharp corrective signal that forces the model to learn the distinction between kinematic events caused by driving maneuvers and those caused by surface roughness. The speed and acceleration context vectors injected via the MLP branch provide the necessary side information for this discrimination.

The full compound loss for a batch is then:

$$\mathcal{L}_{\text{total}} = \mathbb{E}[(\mathcal{L}_{\text{Huber}}(\epsilon) + 0.2 \cdot \mathcal{L}_{\text{LogCosh}}(\epsilon)) \cdot w_{\text{pothole}}(y) \cdot w_{\text{hunter}}(y, v, \bar{a}_y, y)] \quad (32)$$

This formulation is mathematically superior to MSE for this application because it simultaneously achieves robust outlier resistance (Huber/Log-Cosh), minority-class amplification (pothole penalty), and false-alarm suppression (hunter penalty)—none of which MSE can provide.

7) *Model Performance*: The model converges after 21 epochs under early stopping with a patience of 15 epochs and adaptive learning rate reduction (ReduceLROnPlateau, factor 0.5, patience 5). On the held-out test set comprising 13,818 windows:

The MAE of 0.33 m/km is particularly notable in context: the IRI thresholds for road maintenance intervention in most national road authorities are in the range of 3–5 m/km for trigger-level maintenance, meaning our model’s prediction error is an order of magnitude smaller than the decision boundary—a sufficient precision for infrastructure management applications.

TABLE III
MODEL PERFORMANCE ON HELD-OUT TEST SET

Metric	Value
Mean Absolute Error (MAE)	0.3291 m/km
Root Mean Squared Error (RMSE)	1.1799 m/km
R^2 Score	0.7595

The trained model is exported as a TensorFlow Lite file (`iri_background_model.tflite`) with post-training quantization, yielding a model footprint suitable for deployment on Android devices with real-time inference.

Note on Future Work: A Pothole Classification Model is currently under active development as the second major ML component of this system. Leveraging the synchronized dashcam imagery and IMU impulse data collected during simulation and field trials, this model will provide discrete event-level pothole detection and localization, complementing the continuous IRI regression pipeline described above. Results from this component will be reported in a forthcoming publication upon completion of the validation pipeline.

D. Real-Time Distributed Architecture and Intelligent Routing

1) *System Overview and Tier Separation*: The deployment backend implements a three-tier distributed architecture that cleanly separates concerns by data velocity and persistence requirements:

TABLE IV
THREE-TIER DISTRIBUTED ARCHITECTURE

Tier	Technology	Role
Tier 1: Ingestion & Compute	Node.js / Express	HTTP API, business logic, optimized for geospatial queries
Tier 2: Persistent Store	MongoDB (Mongoose)	Road segments, observations
Tier 3: Ephemeral State	Redis	WebSocket sessions, live regression

This separation is architecturally principled. MongoDB is optimized for geospatial queries and persistent document storage, but it is not designed for the sub-millisecond key-value lookups required to track thousands of simultaneously connected WebSocket clients. Redis provides the $O(1)$ set-membership and TTL-keyed expiry primitives that make live session management efficient. Conversely, Redis is an in-memory store—making it inappropriate for durable geospatial road quality data. The two stores are thus architecturally complementary, and neither can substitute for the other without significant performance or reliability degradation.

Graceful degradation is a first-class design constraint: the Redis initialization (`initializeRedis`) explicitly catches connection failures and allows the server to continue operating with reduced real-time capability rather than crashing. The `getRedisClient()` function returns `null` if Redis is unavailable, and all callers perform null-guard checks.

2) *The MongoDB Geospatial Data Model*: The `RoadSegment` document schema is the central persistent entity. Each document represents a unique road segment

identified by a composite key `roadSegmentId`, derived from the OSRM map-matching engine. The schema stores:

- A **GeoJSON LineString** geometry (the road centerline, in WGS-84 coordinates).
- A **GeoJSON Point** center point (for efficient centroid-based proximity queries).
- An `aggregatedQualityScore` in $[0, 3]$, representing the consensus road quality rating derived from multiple crowdsourced observations.
- Patch-length accumulators $len1$, $len2$, $len3$: the total length (in metres) of road within this segment that has been classified as quality 1, 2, and 3 respectively, as reported by smartphone-computed IRI predictions.

The center point is indexed with a **2dsphere** geospatial index, enabling MongoDB's `$near` and `$geoWithin` operators for efficient kilometer-scale proximity queries. The `regionId` field (a Geohash string at precision 6) is indexed for $O(\log n)$ region-filtered queries.

Two-Mode Aggregation: The `aggregationService.aggregateRoadSegment()` method implements a two-mode scoring strategy that prioritizes patch-based data when available. Given patch accumulators ℓ_1, ℓ_2, ℓ_3 and a nominal segment length L_{seg} :

$$\text{Score}_{patch} = \frac{\ell_1 \cdot 1 + \ell_2 \cdot 2 + \ell_3 \cdot 3}{L_{seg}} \quad (33)$$

This is a length-weighted mean severity score. The confidence is computed as a convex combination of coverage ratio and observation density:

$$\text{Confidence}_{patch} = 0.6 \cdot \min\left(\frac{\ell_1 + \ell_2 + \ell_3}{L_{seg}}, 1\right) + 0.4 \cdot \min\left(\frac{N_{obs}}{10}, 1\right) \quad (34)$$

When patch data is insufficient (total patch length < 5 m), the system falls back to an observation-based aggregation with exponential time decay:

$$w_k = e^{-t_k/\tau} \cdot w_{speed}(v_k) \cdot c_k \quad (35)$$

where $\tau = T_{decay}$ (default 24 h), t_k is the observation age in hours, w_{speed} is a speed-based reliability weight (lower speeds yield more precise measurements), and c_k is the map-matching confidence score. The aggregated quality score is the normalized weighted mean:

$$\text{Score}_{obs} = \frac{\sum_k w_k \cdot q_k}{\sum_k w_k} \quad (36)$$

where $q_k \in \{0, 1, 2, 3\}$ is the reported road quality at observation k .

3) **WebSocket Geohashing: Precision Namespaces for Targeted Real-Time Broadcast:** The core architectural challenge of a live road quality map is **targeted broadcast**: when a new observation arrives from a driver at location (ϕ, λ) , we must instantaneously notify only the clients whose map viewports overlap that location—not the entire connected user base, which may number in the thousands.

We solve this via **Geohash-based pub/sub namespaces**. The Geohash algorithm (Niemeyer, 2008) encodes any $(latitude, longitude)$ pair as a compact alphanumeric string by recursively bisecting the geographic bounding box. At precision $P = 6$ (our deployment default), each Geohash cell covers approximately **1.2 km $\times$ 0.61 km**—a spatial granularity well-matched to a typical mobile map viewport.

The `getRegionId(latitude, longitude)` utility function in `geohash.js` encodes any GPS coordinate to a precision-6 Geohash string in $O(1)$ time. This string becomes the **Socket.IO room name** for all clients within that geographic cell.

When a new socket client connects and sends a `join-region` event with its GPS coordinates, the server:

- 1) Computes `regionId = getRegionId(latitude, longitude)`.
- 2) Calls `socket.join(regionId)` — registering the socket in that Socket.IO room.
- 3) Records the socket's current region in Redis under the key `session:{socketId}`.

When the client moves to a new location (via `update-location` events, broadcast at each GPS poll), the server computes the new Geohash and compares it to the socket's current region. If they differ, the socket silently leaves the old room and joins the new one:

```
socket.leave(oldRegionId)    -> O(1) hash table rehash
socket.join(newRegionId)     -> O(1) hash table insert
```

This continuous region tracking means that **every broadcast is $O(R)$** where R is the number of clients in the target region—which is typically a small fraction of the total connected client pool. The server never iterates over all connected sockets to determine broadcast targets; Socket.IO's room mechanism uses internal hash-map lookups.

Neighbor Region Awareness: A single Geohash cell does not capture the full viewport of a moving map. The `getNeighbors(regionId)` function returns the 9-cell neighborhood (center + 8 adjacent cells) of any Geohash, covering the ~ 5 km $\times$ 3 km area centered on the user. This 9-cell set is used in the REST API for road segment queries (`getRoadSegmentsByRegion`), ensuring that road data at viewport edges is never clipped.

Broadcast Threshold: The `shouldBroadcastUpdate(oldScore, newScore, confidenceScore)` function enforces two broadcast conditions before emitting a road-quality-update event: (1) the confidence score must exceed 0.5 (suppressing low-confidence updates that could cause flickering in the client map), and (2) the score must have changed by more than 0.3 on the 0–3 scale. This threshold filtering prevents the server from generating broadcast storms when many observations arrive simultaneously for the same segment, as is common at traffic lights.

Ephemeral Session State in Redis: Every active WebSocket session is stored in Redis under the key `session:{socketId}` with a 5-minute TTL. The value is

a JSON document recording the socket's current `regionId`, last known location, and connection metadata. Redis set operations under `region:{regionId}:members` track which sockets are in each geographic cell—enabling the server to determine region occupancy without iterating over all socket rooms. All session keys self-expire after 5 minutes of inactivity (enforced by the heartbeat TTL), ensuring that Redis memory footprint is bounded even during client crashes that prevent clean disconnect events.

4) *The Ride Comfort Score and OSRM-Based Infrastructure-Aware Routing*: The routing service (routeScoringService) elevates the system beyond a mere data visualization tool to an **active infrastructure-aware navigation assistant**. The API endpoint `GET /api/routes/score` accepts a source and destination, retrieves up to three candidate routes from the Mapbox Directions API, scores each for road quality, and returns a ranked recommendation.

Route Geometry Sampling: Each Mapbox route is returned as a polyline6-encoded LineString. This geometry is decoded and sub-sampled at approximately every $d_{step} = 75$ m along the route using a linear interpolation walk. The resulting set of M sample points $\{(\phi_i, \lambda_i)\}_{i=1}^M$ constitutes the spatial skeleton of the route.

In-Memory Nearest-Segment Matching: Rather than issuing M individual MongoDB queries (one per sample point), the service performs a single bulk query to prefetch all road segments from the relevant Geohash regions across all candidate routes:

```
await RoadSegment.find({ regionId: { $in: regionIds } })
```

The resulting candidate set is held in memory, and each sample point is matched to its nearest candidate segment by brute-force minimum Haversine distance search over the in-memory array. A match is accepted only if the nearest segment lies within $r_{max} = 120$ m; otherwise, the point is scored with a default quality value of 1.0 m/km (representing the global average road quality under missing data).

Distance-Weighted Quality Score: The road quality score for each route is the distance-weighted mean of matched segment scores:

$$Q_{route} = \frac{\sum_{e \in \text{edges}} q_e \cdot \ell_e}{\sum_{e \in \text{edges}} \ell_e} \quad (37)$$

where q_e is the matched quality score at the endpoint of edge e and ℓ_e is the edge length in metres. This weighting ensures that short high-quality segments between long degraded stretches do not inappropriately inflate the score.

Composite Route Ranking: Routes are ranked by a composite score that balances quality against distance efficiency:

$$S_{final} = \underbrace{\frac{Q_{route}}{3}}_{\text{normalized quality}} \cdot w_Q + \underbrace{\frac{d_{route} - d_{min}}{d_{max} - d_{min}}}_{\text{normalized excess distance}} \cdot w_D + \text{Penalty}_{\text{overage}} \quad (38)$$

where $w_Q = 0.7$ and $w_D = 0.3$ encode a preference for road quality over distance minimization, and a penalty of 0.35 is added to any route whose length exceeds $1.8 \times d_{min}$ to prevent recommending unreasonably circuitous routes despite high quality.

Redis-Backed Route Caching with Semantic Freshness Checks: Route computation is expensive (Mapbox API call + bulk MongoDB query + M-point matching). Results are cached in Redis under a key derived from the source/destination coordinates (rounded to 4 decimal places, ~ 11 m precision) and the number of requested alternatives, with a 15-minute TTL.

Crucially, the cache is not invalidated purely on time. On each cache read, the service performs a **semantic freshness check**: it retrieves the cached fingerprint—the top- $K = 8$ road segment IDs that contributed most to the best route's score, each annotated with their score snapshot at cache time—and queries MongoDB for their current scores. If any segment's score has changed by more than $\Delta_{threshold} = 0.25$ since caching, the cache entry is invalidated and the route is recomputed. This ensures that a road segment freshly reported as severely damaged (e.g., after a pothole report) will propagate into route recommendations within seconds of the next user query, rather than being masked by a 15-minute stale cache.

This combination of geohash-based spatial namespacing, in-memory geometry matching, composite scoring, and semantically-aware caching constitutes a full stack infrastructure-aware routing system that, to the authors' knowledge, represents one of the most complete real-time crowd-sourced road quality routing implementations reported in the academic literature.

REFERENCES

- [1] Ministry of Road Transport and Highways (MoRTH), "Basic Road Statistics of India," Transport Research Wing, Government of India, New Delhi, Tech. Rep., 2023.
- [2] World Bank, "Guide for Road Safety Opportunities and Challenges: Low- and Middle-Income Countries Profiles," World Bank Group, Washington, DC, Tech. Rep., 2019.
- [3] Sayers, Gillespie, and Paterson, "Guidelines for Conducting and Calibrating Road Roughness Measurements," World Bank, Washington, DC, Tech. Rep. World Bank Technical Paper No. 46, 1986.
- [4] R. Ganti, F. Ye, and H. Lei, "Mobile crowdsensing: current state and future challenges," *IEEE Communications Magazine*, vol. 49, no. 11, pp. 32–39, Nov. 2011. [Online]. Available: <http://ieeexplore.ieee.org/document/6069707/>
- [5] P. Mohan, V. N. Padmanabhan, and R. Ramjee, "Nericell: rich monitoring of road and traffic conditions using mobile smartphones," in *Proceedings of the 6th ACM conference on Embedded network sensor systems*. Raleigh NC USA: ACM, Nov. 2008, pp. 323–336. [Online]. Available: <https://dl.acm.org/doi/10.1145/1460412.1460444>
- [6] R. Kumar, A. Mukherjee, and V. P. Singh, "Community Sensor Network for Monitoring Road Roughness Using Smartphones," *Journal of Computing in Civil Engineering*, vol. 31, no. 3, p. 04016059, May 2017. [Online]. Available: <https://ascelibrary.org/doi/10.1061/%28ASCE%29CP.1943-5487.0000624>
- [7] S. Sattar, S. Li, and M. Chapman, "Road Surface Monitoring Using Smartphone Sensors: A Review," *Sensors*, vol. 18, no. 11, p. 3845, Nov. 2018. [Online]. Available: <https://www.mdpi.com/1424-8220/18/11/3845>
- [8] S. Alqaydi, W. Zeiada, A. El Wakil, A. J. Alnaqbi, and A. Azam, "A Comprehensive Review of Smartphone and Other Device-Based Techniques for Road Surface Monitoring," *Eng*, vol. 5,

- no. 4, pp. 3397–3426, Dec. 2024. [Online]. Available: <https://www.mdpi.com/2673-4117/5/4/177>
- [9] N. Goodall, “Evaluation of Crowdsourced Data on Unplowed Roads,” Oct. 2023, arXiv:2311.10740 [cs]. [Online]. Available: <http://arxiv.org/abs/2311.10740>
- [10] A. S. El-Wakeel, A. Noureldin, H. S. Hassanein, and N. Zorba, “iDriveSense: Dynamic Route Planning Involving Roads Quality Information,” in *2018 IEEE Global Communications Conference (GLOBECOM)*, Dec. 2018, pp. 1–6, iSSN: 2576-6813. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8648009>
- [11] J. Eriksson, L. Girod, B. Hull, R. Newton, S. Madden, and H. Balakrishnan, “The pothole patrol: using a mobile sensor network for road surface monitoring,” in *Proceedings of the 6th international conference on Mobile systems, applications, and services*, ser. MobiSys ’08. New York, NY, USA: Association for Computing Machinery, Jun. 2008, pp. 29–39. [Online]. Available: <https://dl.acm.org/doi/10.1145/1378600.1378605>
- [12] D. K. Dewangan and S. P. Sahu, “PotNet: Pothole detection for autonomous vehicle system using convolutional neural network,” *Electronics Letters*, vol. 57, no. 2, pp. 53–56, 2021, eprint: <https://ietresearch.onlinelibrary.wiley.com/doi/pdf/10.1049/ell2.12062>. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1049/ell2.12062>
- [13] S. Balakuntala and S. Venkatesh, “An Intelligent System to Detect, Avoid and Maintain Potholes: A Graph Theoretic Approach,” Sep. 2013, arXiv:1305.5522 [cs]. [Online]. Available: <http://arxiv.org/abs/1305.5522>
- [14] A. Mednis, G. Strazdins, R. Zviedris, G. Kanonirs, and L. Selavo, “Real time pothole detection using Android smartphones with accelerometers,” in *2011 International Conference on Distributed Computing in Sensor Systems and Workshops (DCOSS)*, Jun. 2011, pp. 1–6, iSSN: 2325-2944. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/5982206>
- [15] BeamNG GmbH, “BeamNG.tech,” Bremen, Germany, 2026. [Online]. Available: <https://beamng.tech/>